МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №4
по дисциплине «Объектно-ориентированное проргаммирование»
Тема: Реализация игры с пошаговым перемещением игрока и сражением с врагами.

Студент гр. 4381 _____ Захаренко А.В.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2025

Цель работы.

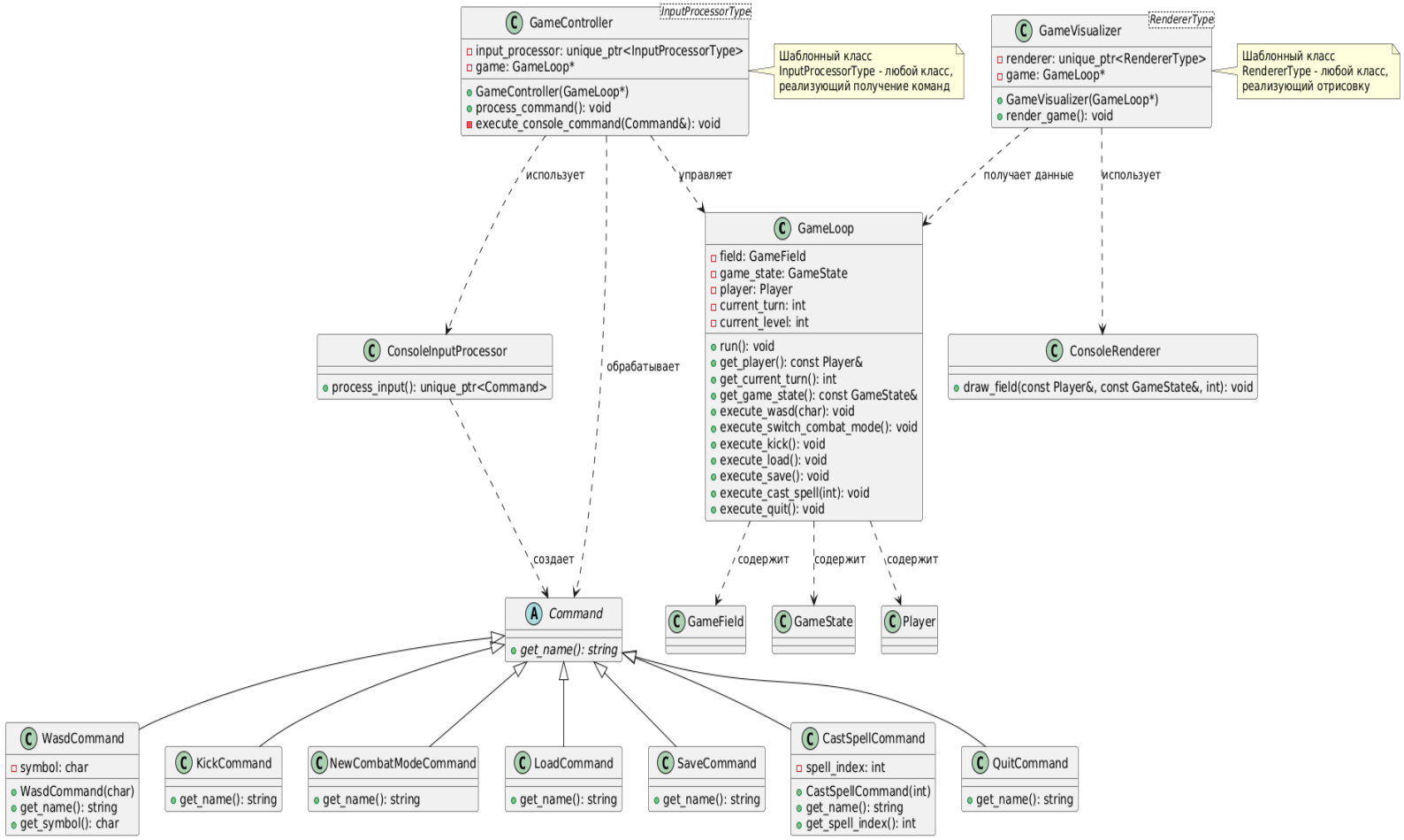
Продолжить реализовывать игру с пошаговым перемещением игрока и сражением с врагами. Расширить существующую реализацию согласно ТЗ: создать два шаблонных класса (управление игрой и отрисовка), написать конктретные реализации (в данном случае консольные). Особое внимание уделить тому, что ввод преобразуется в объект. И все это вместе интегрировать в игровой цикл, тем самым получив финальную простейщую реализацию игры (цель всех 4 л/р).

Задание.

На 6/3/1 баллов:

1. Создать класс считывающий ввод пользователя и преобразующий ввод пользователь в объект команды.
2. Создать класс отрисовки игры. Данный класс определяет то, как должно отображаться игра.
3. Создать шаблонный класс управления игрой. В качестве параметра шаблона должен передаваться класс, отвечающий за считывание и преобразование ввода. У себя он создает объект класса из параметра шаблона и получает от него команды, а далее вызывает нужное действие у классов игры. Данный класс не должен создавать объект класса игры. Реализация должна быть такой, что можно масштабировать программу, например, реализовать получение команд через интернет без использования реализации интерфейса, и просто подставить новый класс в качестве параметра шаблона.
4. Создать шаблонный класс визуализации игры. . В качестве параметра шаблона должен передаваться класс, отвечающий за способ отрисовки игры. Данный класс создает объект класса отрисовки игры, и реагирует на изменения в игре, и вызывает команду отрисовку. Реализация должна быть такой, что можно масштабировать программу, например, реализовать отрисовку в виде веб-страницы без использования реализации интерфейса, и просто подставить новый класс в качестве параметра шаблона.

UML-диаграмма новых классов и зависимостей.



Выполнение работы.

Расспишем работу программы: реализацию новых класов, их назначение и основные архитектурные решения.

1) Реализация системы комманд.

Классы системы команд согласно нажатым клавишам в данной реализации:

- Command - абстрактный базовый класс всех команд - Для потенциальной расширяемости системы комманд было принято решение обобщить команду абстрактным классом и от него уже насследовать конкретные команды.
- WasdCommand - команда движения (W, A, S, D)
- KickCommand - команда атаки (E)
- NewCombatModeCommand - команда смены режима боя (F)
- LoadCommand - команда загрузки игры (L)
- SaveCommand - команда сохранения игры (V)
- CastSpellCommand - команда использования заклинания (0-4)
- QuitCommand - команда выхода из игры (Q)

Методы классов команд:

- get_name() - возвращает строковое представление команды
- get_symbol() (в WasdCommand) - возвращает символ движения (уточнение команды)
- get_spell_index() (в CastSpellCommand) - возвращает индекс заклинания (уточнение команды)

2) Преобразование ввода из консоли и шаблонное управление игрой.

Ранее обработка ввода команды пользователем была по ходу игры была в основном цикле и классе GameLoop, однако необходимо соответсвовать принципу разделения ответственности и поэтому данная логика была вынесена. А также шаблонная реализация упрощает потенциальное масштабирование.

Класс **ConsoleInputProcessor** - преобразователь консольного ввода, реализует чтение ввода с консоли и преобразование символов в объекты команд

Методы:

- process_input() - основной метод, считывающий символ и возвращающий соответствующую команду (экземпляр класса соответсвующей команды согласно введенному символу)

Класс **GameController<InputProcessorType>** - шаблонный контроллер игры, реализует управление потоком команд между системой ввода и игровой логикой.

Поля:

- input_processor - умный указатель (unique_ptr) на объект обработки ввода
- game - указатель на игровой цикл

Методы:

- GameController(GameLoop*) - конструктор, инициализирующий обработчик ввода
- process_command() - основной метод обработки одной команды
- execute_console_command(Command&) - метод для выполнения команд (считывание + выполнение логики из)

3) Консольный отрисовщик и шаблонный класс отрисовки.

Ранее отрисовка поля игры была также реализована в основном цикле игры, что могло перегружать его ответсвенность. Поэтому очень разумно это все вынести.

Класс **ConsoleRenderer** - консольный отрисовщик для визуализации игрового состояния в текстовом виде в консоли

Методы:

- draw_field(const Player&, const GameState&, int) - основной метод отрисовки

Класс **GameVisualizer<RendererType>** - шаблонный визуализатор для управления процессом отрисовки через абстракцию рендерера

Поля:

- renderer - умный указатель на объект рендерера
- game - указатель на игровой цикл

Методы:

- GameVisualizer(GameLoop*) - конструктор, создающий рендерер через std::make_unique
- render_game() - основной метод, вызывающий отрисовку текущего состояния

Вывод.

Благодаря реализации двух шаблонных классов (управление игроком и отрисовка игры) программа более хорошо подлежит масштабированию. Можно написать не консольные реализации (CLI), а GUI и подставить эти классы в параметр шаблона. Цели данной л/р можно считать достигнутыми. И в целом финальный продукт всех 4 л/р функционирует: игра запускается, ошибок в логике хода игры при тестировании обнаружено не было, функционирует быстро, визуализация приемлемая и содержит много полезного справочного вывода (что-то по типу логирования), что тоже можно считать успехом.